Definitions in `Sarason/` and `R2.lean`

Lean Formalization Project

September 10, 2026

This document provides a comprehensive list of all definitions manually formalized in the `ComplexAnalysis` project, specifically within `R2.lean` and the `Sarason/` directory. For each definition, the raw Lean code is presented first, followed by its mathematical interpretation.

1 ComplexAnalysis/R2.lean

- `sqNorm`

```
noncomputable def sqNorm (x : ℝ × ℝ) : ℝ := x.1^2 + x.2^2
```

$$\text{sqNorm}(x) = x_1^2 + x_2^2$$

- `sqDist`

```
noncomputable def sqDist (x y : ℝ × ℝ) : ℝ := sqNorm (x - y)
```

$$\text{sqDist}(x, y) = \text{sqNorm}(x - y)$$

- `euclideanNorm`

```
noncomputable def euclideanNorm (x : ℝ × ℝ) : ℝ := Real.sqrt (sqNorm x)
```

$$\text{euclideanNorm}(x) = \sqrt{x_1^2 + x_2^2}$$

- `euclideanDist`

```
noncomputable def euclideanDist (x y : ℝ × ℝ) : ℝ := Real.sqrt (sqDist x y)
```

$$\text{euclideanDist}(x, y) = \sqrt{\text{sqDist}(x, y)}$$

- `LimitRtoR2`

```
def LimitRtoR2 (f : ℝ → ℝ × ℝ) (a : ℝ) (L : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - a| ∧ |x - a| < δ → euclideanDist (f x) L < ε
```

$$\lim_{x \to a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x, 0 < |x - a| < \delta \implies \text{euclideanDist}(f(x), L) < \epsilon$$

- `LimitR2toR`

```
def LimitR2toR (f : ℝ × ℝ → ℝ) (a : ℝ × ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ × ℝ, 0 < euclideanDist x a ∧ euclideanDist x a < δ → |
    f x - L| < ε
```

$$\lim_{x \to a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x \in \mathbb{R}^2, 0 < \text{euclideanDist}(x, a) < \delta \implies |f(x) - L| < \epsilon$$

- LimitR2toR2

```
def LimitR2toR2 (f : ℝ × ℝ → ℝ × ℝ) (a : ℝ × ℝ) (L : ℝ × ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ × ℝ, 0 < euclideanDist x a ∧ euclideanDist x a < δ →
    euclideanDist (f x) L < ε
```

$$\lim_{x \to a} f(x) = L \iff \forall \epsilon > 0, \exists \delta > 0, \forall x \in \mathbb{R}^2, 0 < \text{euclideanDist}(x, a) < \delta \implies \text{euclideanDist}(f(x), L) < \epsilon$$

- ConvergesR2

```
def ConvergesR2 (seq : ℕ → ℝ × ℝ) (L : ℝ × ℝ): Prop :=
  ∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, euclideanDist (seq n) L < ε
```

$$\lim_{n \to \infty} \text{seq}(n) = L \iff \forall \epsilon > 0, \exists N \in \mathbb{N}, \forall n \geq N, \text{euclideanDist}(\text{seq}(n), L) < \epsilon$$

- HasFDerivAt_R2_eps

```
def HasFDerivAt_R2_eps (u v : ℝ × ℝ → ℝ) (ux0 uy0 vx0 vy0 : ℝ) (a : ℝ × ℝ) :
    Prop :=
  LimitR2toR2 (fun h => (
    (u (a.1 + h.1, a.2 + h.2) - u a) - (ux0 * h.1 + uy0 * h.2),
    (v (a.1 + h.1, a.2 + h.2) - v a) - (vx0 * h.1 + vy0 * h.2)
  )) (0, 0) (0, 0)
```

$$\lim_{h \to 0} \left(F(a+h) - F(a) - \begin{pmatrix} u_{x0} & u_{y0} \\ v_{x0} & v_{y0} \end{pmatrix} h \right) = 0$$

- DifferentiableAt_R2_eps

```
def DifferentiableAt_R2_eps (u v : ℝ × ℝ → ℝ) (a : ℝ × ℝ) : Prop :=
  ∃ ux0 uy0 vx0 vy0 : ℝ, HasFDerivAt_R2_eps u v ux0 uy0 vx0 vy0 a
```

$$\exists u_{x0}, u_{y0}, v_{x0}, v_{y0} \in \mathbb{R}, \text{HasFDerivAt\_R2\_eps}(u, v, u_{x0}, u_{y0}, v_{x0}, v_{y0}, a)$$

- HasDerivAt_RtoR2_eps

```
def HasDerivAt_RtoR2_eps (f : ℝ → ℝ × ℝ) (f'0 : ℝ × ℝ) (t₀ : ℝ) : Prop :=
  LimitRtoR2 (fun t => (
    (f t).1 - (f t₀).1 - (t - t₀) * f'0.1,
    (f t).2 - (f t₀).2 - (t - t₀) * f'0.2
  )) t₀ (0, 0)
```

$$\lim_{t \to t_0} (f(t) - f(t_0) - (t - t_0) f_0') = 0$$

2 ComplexAnalysis/Sarason/Definitions.lean

- HasLimitAt_eps

```
def HasLimitAt_eps (f : ℂ → ℂ) (L : ℂ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ z : ℂ, 0 < Complex.abs (z - z₀) ∧ Complex.abs (z - z₀) < δ →
     Complex.abs (f z - L) < ε
```

$$\lim_{z \to z_0} f(z) = L$$

- HasLimitAtInf_eps

```
def HasLimitAtInf_eps (f : ℂ → ℝ) (L : ℝ) : Prop :=
  ∀ ε > 0, ∃ R > 0, ∀ z : ℂ, Complex.abs z > R → |f z - L| < ε
```

$$\lim_{|z|\to\infty} f(z) = L$$

- HasDerivAt_eps

```
def HasDerivAt_eps (f : ℂ → ℂ) (f' : ℂ) (z₀ : ℂ) : Prop :=
  HasLimitAt_eps (fun z => (f z - f z₀) / (z - z₀)) f' z₀
```

$$\lim_{z\to z_0} \frac{f(z) - f(z_0)}{z - z_0} = f'$$

- DifferentiableAt_eps

```
def DifferentiableAt_eps (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  ∃ f', HasDerivAt_eps f f' z₀
```

f is differentiable at z_0

- HasPartialDerivX_C_to_R_eps

```
def HasPartialDerivX_C_to_R_eps (u : ℂ → ℝ) (ux : ℝ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ x : ℝ, 0 < |x - z₀.re| ∧ |x - z₀.re| < δ →
    |(u ((x : ℂ) + (z₀.im : ℂ) * I) - u z₀) / (x - z₀.re) - ux| < ε
```

$$\frac{\partial u}{\partial x}(z_0) = u_x$$

- HasPartialDerivY_C_to_R_eps

```
def HasPartialDerivY_C_to_R_eps (u : ℂ → ℝ) (uy : ℝ) (z₀ : ℂ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ y : ℝ, 0 < |y - z₀.im| ∧ |y - z₀.im| < δ →
    |(u ((z₀.re : ℂ) + (y : ℂ) * I) - u z₀) / (y - z₀.im) - uy| < ε
```

$$\frac{\partial u}{\partial y}(z_0) = u_y$$

- deriv_eps

```
noncomputable def deriv_eps (f : ℂ → ℂ) (z₀ : ℂ) (h : DifferentiableAt_eps f z
    ₀) : ℂ :=
  Classical.choose h
```

$$\text{deriv\_eps}(f, z_0) = f'(z_0)$$

- HasDerivAt_R_to_C_eps

```
def HasDerivAt_R_to_C_eps (f : ℝ → ℂ) (f'0 : ℂ) (t₀ : ℝ) : Prop :=
  ∀ ε > 0, ∃ δ > 0, ∀ t : ℝ, 0 < |t - t₀| ∧ |t - t₀| < δ →
    Complex.abs ((f t - f t₀) / (t - t₀) - f'0) < ε
```

$$\lim_{t\to t_0} \frac{f(t) - f(t_0)}{t - t_0} = f_0'$$

- HolomorphicAt_eps

```
def HolomorphicAt_eps (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  ∃ r > 0, ∀ z, Complex.abs (z - z₀) < r → DifferentiableAt_eps f z
```

f is holomorphic at $z_0 \iff f$ is complex-differentiable on some open ball around z_0

- HolomorphicOn_eps

```
def HolomorphicOn_eps (f : ℂ → ℂ) (G : Set ℂ) : Prop :=
  ∀ z ∈ G, HolomorphicAt_eps f z
```

f is holomorphic on G

3 ComplexAnalysis/Sarason/Chapter1.lean

- chordalMetric

```
noncomputable def chordalMetric (z w : ℂ) : ℝ :=
  (2 * Complex.abs (z - w)) / (Real.sqrt (1 + Complex.normSq z) * Real.sqrt (1
    + Complex.normSq w))
```

$$\chi(z,w) = \frac{2|z-w|}{\sqrt{1+|z|^2}\sqrt{1+|w|^2}}$$

- chordalMetricToInf

```
noncomputable def chordalMetricToInf (z : ℂ) : ℝ :=
  2 / Real.sqrt (1 + Complex.normSq z)
```

$$\chi(z,\infty) = \frac{2}{\sqrt{1+|z|^2}}$$

- Φ (Stereographic Projection)

```
noncomputable def Φ (z : ℂ) : EuclideanSpace ℝ (Fin 3) :=
  ![ (2 * z.re) / (1 + Complex.normSq z),
     (2 * z.im) / (1 + Complex.normSq z),
     (Complex.normSq z - 1) / (1 + Complex.normSq z) ]
```

$$\Phi(z) = \left(\frac{2\,\mathrm{Re}(z)}{1+|z|^2}, \frac{2\,\mathrm{Im}(z)}{1+|z|^2}, \frac{|z|^2-1}{1+|z|^2}\right) \in S^2$$

- ChordalC

```
def Chordalℂ : Type := ℂ
```

ChordalC $= \mathbb{C}$ (equipped with chordal metric)

4 ComplexAnalysis/Sarason/Chapter2.lean

- del (∂)

```
def del (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  (1/2 : ℂ) * (deriv (fun x : ℝ ⇒ f (z + x)) 0 - I * deriv (fun y : ℝ ⇒ f (z
    + y * I)) 0)
```

$$\partial f = \frac{1}{2}\left(\frac{\partial f}{\partial x} - i\frac{\partial f}{\partial y}\right)$$

- delBar ($\bar{\partial}$)

```
def delBar (f : ℂ → ℂ) (z : ℂ) : ℂ :=
  (1/2 : ℂ) * (deriv (fun x : ℝ ⇒ f (z + x)) 0 + I * deriv (fun y : ℝ ⇒ f (z
    + y * I)) 0)
```

$$\bar{\partial} f = \frac{1}{2}\left(\frac{\partial f}{\partial x} + i\frac{\partial f}{\partial y}\right)$$

- del_eps

```
noncomputable def del_eps (ux0 uy0 vx0 vy0 : ℝ) : ℂ :=
  ((ux0 + vy0) / 2 : ℝ) + I * ((vx0 - uy0) / 2 : ℝ)
```

$$\partial f = \frac{1}{2}(u_{x0} + v_{y0}) + \frac{i}{2}(v_{x0} - u_{y0})$$

- delBar_eps

```
noncomputable def delBar_eps (ux0 uy0 vx0 vy0 : ℝ) : ℂ :=
  ((ux0 - vy0) / 2 : ℝ) + I * ((vx0 + uy0) / 2 : ℝ)
```

$$\bar{\partial} f = \frac{1}{2}(u_{x0} - v_{y0}) + \frac{i}{2}(v_{x0} + u_{y0})$$

- path_in_C

```
def path_in_C (x y : ℂ) : Type := Path x y
```

$$\gamma : [0,1] \to \mathbb{C} \text{ where } \gamma(0) = x, \gamma(1) = y$$

- translated_path

```
noncomputable def translated_path (c x y : ℂ) (γ : path_in_C x y) : path_in_C
    (c + x) (c + y) where
  toFun := fun t => c + γ t
```

$$\gamma_c(t) = c + \gamma(t)$$

- pathDeriv

```
noncomputable def pathDeriv {x y : ℂ} (γ : path_in_C x y) (t : ℝ) : ℂ :=
  deriv γ.extend t
```

$$\gamma'(t) = \frac{d\gamma}{dt}$$

- pathDirection

```
noncomputable def pathDirection {x y : ℂ} (γ : path_in_C x y) (t : ℝ) : ℝ :=
  Complex.arg (pathDeriv γ t)
```

$$\text{Direction}(\gamma, t) = \arg(\gamma'(t))$$

- pathAngle

```
noncomputable def pathAngle {x₁ y₁ x₂ y₂ : ℂ} (γ₁ : path_in_C x₁ y₁) (t₁ : ℝ)
  (γ₂ : path_in_C x₂ y₂) (t₂ : ℝ) : ℝ :=
  pathDirection γ₂ t₂ - pathDirection γ₁ t₁
```

$$\text{Angle}(\gamma_1, t_1, \gamma_2, t_2) = \arg(\gamma_2'(t_2)) - \arg(\gamma_1'(t_1))$$

- conformal

```
def conformal (f : ℂ → ℂ) (z₀ : ℂ) : Prop :=
  ∀ x₁ y₁ x₂ y₂ : ℂ, ∀ γ₁ : path_in_C x₁ y₁, ∀ t₁ : ℝ,
  ∀ γ₂ : path_in_C x₂ y₂, ∀ t₂ : ℝ,
  γ₁.extend t₁ = z₀ → γ₂.extend t₂ = z₀ →
  DifferentiableAt ℝ γ₁.extend t₁ → DifferentiableAt ℝ γ₂.extend t₂ →
  pathDeriv γ₁ t₁ ≠ 0 → pathDeriv γ₂ t₂ ≠ 0 →
  DifferentiableAt ℝ (f ∘ γ₁.extend) t₁ → DifferentiableAt ℝ (f ∘ γ₂.extend) t
    ₂ →
```

```
  pathDeriv (f_circ_gamma γ₁ f) t₁ ≠ 0 ∧
  pathDeriv (f_circ_gamma γ₂ f) t₂ ≠ 0 ∧
  pathAngle (f_circ_gamma γ₁ f) t₁ (f_circ_gamma γ₂ f) t₂ = pathAngle γ₁ t₁ γ₂ t
    ₂
```

Conformal f at $z_0 \iff f$ preserves angles between all regular intersecting curves at z_0

- centered_line_path

```
noncomputable def centered_line_path (z v : ℂ) : path_in_C (z - v) (z + v)
    where
  toFun := fun t => z + (2 * (t : ℝ) - 1) * v
```

$$\gamma(t) = z + (2t - 1)v$$

5 ComplexAnalysis/Sarason/Chapter3.lean

- translationMatrix

```
noncomputable def translationMatrix (b : ℂ) : GL (Fin 2) ℂ :=
  Units.mk0 !![1, b; 0, 1] ...
```

$$\begin{pmatrix} 1 & b \\ 0 & 1 \end{pmatrix}$$

- scalingMatrix

```
noncomputable def scalingMatrix (a : ℂ) (ha : a ≠ 0) : GL (Fin 2) ℂ :=
  Units.mk0 !![a, 0; 0, 1] ...
```

$$\begin{pmatrix} a & 0 \\ 0 & 1 \end{pmatrix}$$

- inversionMatrix

```
noncomputable def inversionMatrix : GL (Fin 2) ℂ :=
  Units.mk0 !![0, 1; 1, 0] ...
```

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$